University of Toronto

CSC490

A3

Name	Student Number
Yusuf Moola	1007658777
Mikhail Skazhenyuk	1009376337
Rudraksh Monga	1008018342
Codi Lee	1007698608

1 Diagramming Infrastructure

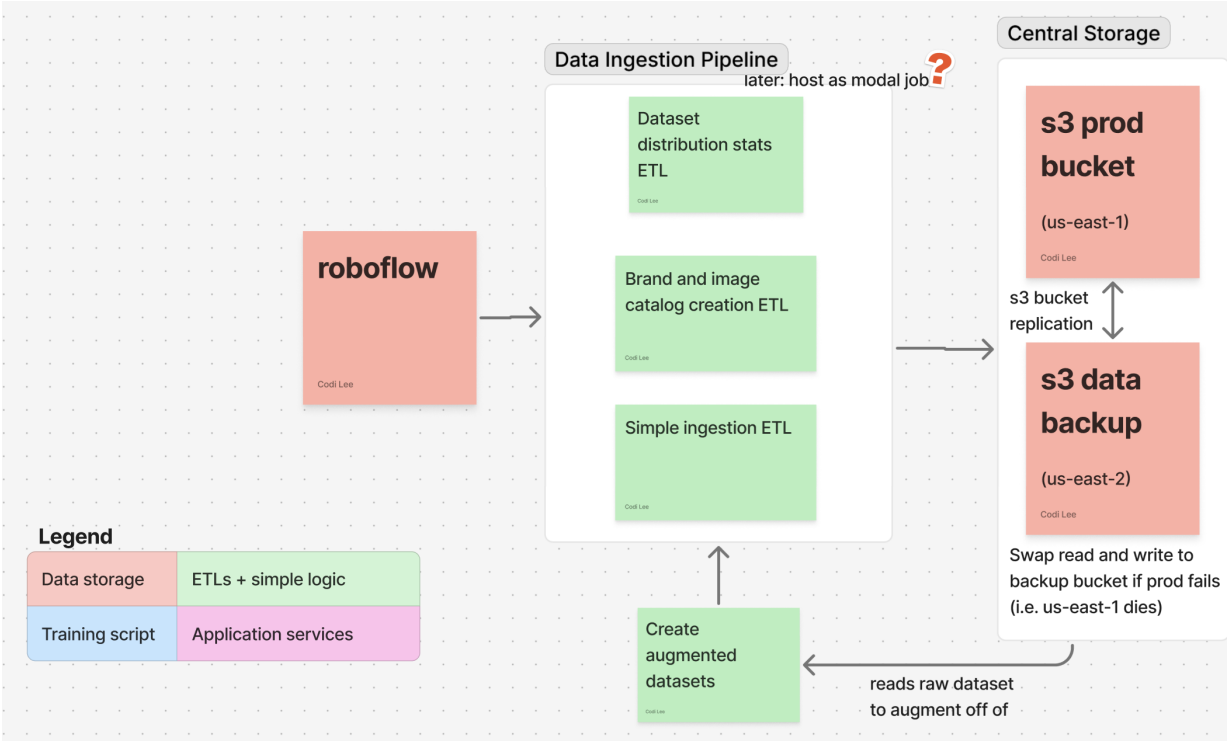

Figure 1: The data processing and ingestion pipeline half of the infrastructure. Red includes data storages which the green ETLs and simple logic modify and save back into the storage buckets. The ingestion ETLs currently run locally, but it is better to transfer them into Modal jobs to handle larger dataset ingestions. Notably, the prod S3 bucket has a replication backup with ETL logic that swaps reading destination in case of prod S3 bucket downage.

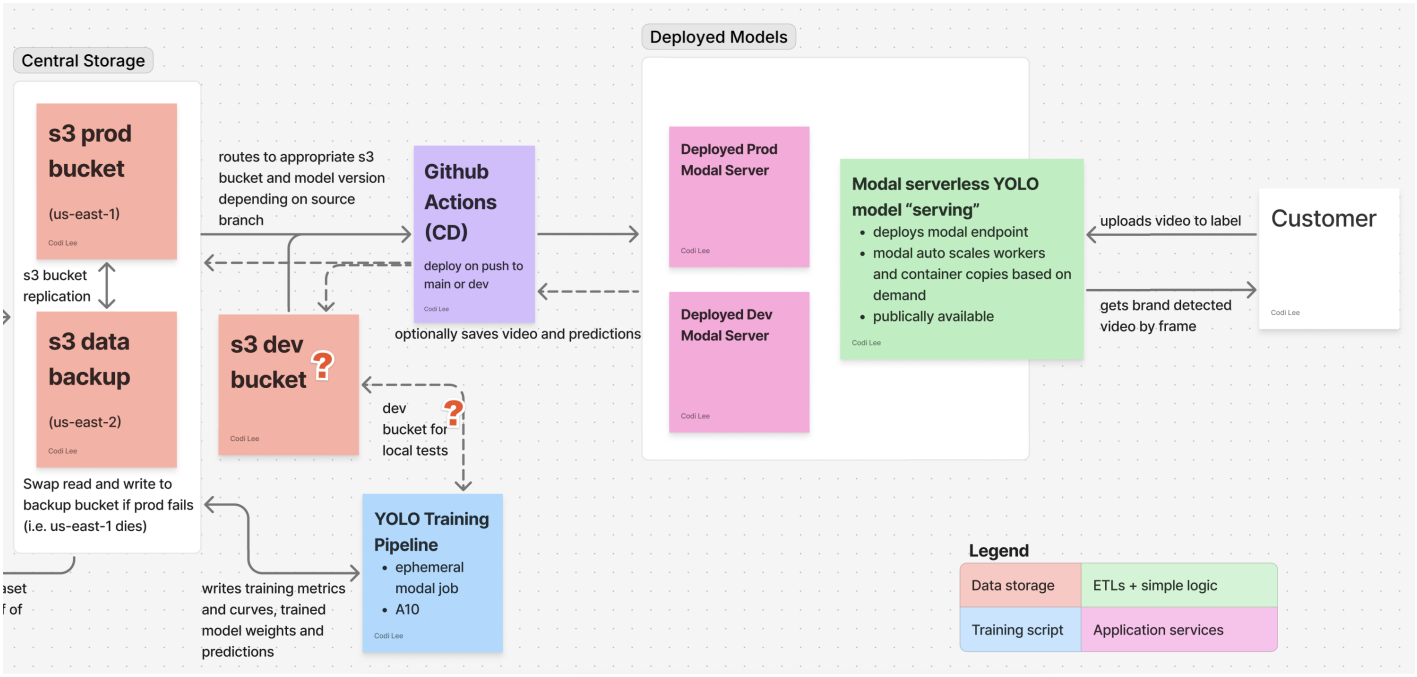

Figure 2: The model training and deploy half of the infrastructure and logic. The training pipeline reads and writes to appropriate S3 buckets to train the YOLO model. This is an ephemeral Modal job. There is a Modal app for deploying the trained model. This diagram treats the app's script as an IaC configuration since this base code is actually deployed with two copies: dev and prod. This allows the server to be tested freely before any changes might disrupt prod traffic. The Github Actions mediates deployments to each dev and prod application servers automatically as triggered by pushes on github to the dev or main branches. The customer interfaces with the endpoints on the servers; passing in videos to the endpoints and retrieving frame-by-frame labeled responses. The dev S3 bucket has yet to be created. This is more of a conceptual idea for this projects' hypothetical startup.

2 Infrastructure as Code

We implement a non-traditional IaC approach as our infrastructure centers around Modal's serverless platform. We chose Modal since our startup's expected traffic is fairly unpredictable throughout the day, but should be ready at a moment's notice (i.e. scattered select customers from around the world who buy access to the application server). During more busy times of the year in the F1 season, we may consider switching model serving approach to something more continuous.

Like IaC frameworks, Modal simplifies the deployment process and abstracts away much of the details for preparing and reloading the infrastructure. While Modal is more extreme in the level of abstraction, it uses Python function and class decorators to specify the configurations of each job and server and ensure consistency and reproducibility.

Below, we list integrations and use cases for our infrastructure, citing the corresponding locations in the diagram and in the code.

2.1 Authentication and Secrets

- Authentication is required for Modal's API key for any compute that needs to run scripts remotely using Modal containers. For local development and deployment, the local terminal will remember the API key and secret after signing in once using the Modal token set command.
- GitHub actions deploys the model application servers and requires a Modal key and secret to be stored in GitHub secrets.
- Modal secrets store S3 API keys and secrets to properly "mount" containers onto the buckets. These can be added on the webpage or in the terminal.
- When using boto3, the local device's .aws/credentials are used to authenticate.

Figure 2 Github Actions CD uses these authentications. Any usage of Modal uses the Modal authentications and secrets. For an example of Modal accessing S3 secrets, see line 27 in file modal/training.py on branch modal_integration_training.

2.2 Reading from and Writing to S3 Buckets

- Modal can be mounted onto an S3 bucket for reading and writing. This bucket must be specified at the creation of the container for a python function.
- Boto3 can also be used to read and write to an S3 bucket. Boto3 can also be used within a Modal deployment to access a bucket that the container is not mounted on.

Figure 1's data ingestion pipeline uses boto3 to read and write to S3. This logic is localized in our code with an s3Client class: pipelines/s3_client.py on the data_ingestion_first_pass_local branch.

Figure 2's Modal jobs and deployments mount onto an S3 bucket to read and write. Line 229 within the Modal function decorator in file modal/training.py on branch modal_integration_training.

Figures 1 and 2's selection between prod and backup buckets are done by passing in multiple bucket options into the s3Client, which chooses the first one that does not fail the boto3 connection. File modal/training.py on branch modal_integration_training also naively implements this by duplicating the modal function and running the backup on failure of the primary bucket. As a potential future scope, we could instead use Terraform to more cleanly handle falling back to S3 backup data.

2.3 Building and maintaining containers

At high level, Modal allows for both ephemeral jobs which only exist to complete a particular compute task (ex. training script: modal/training.py on branch modal_integration_training) and serverless deployment which can quickly spin up containers as needed when the endpoint is called but doesn't require resources otherwise (ex. inference deployment service: pipelines/inference/model_server.py on branch deployed_inference).

Modal allows for container configurations to be standardized by the Modal image (which installs libraries) and function decorators which describes timeouts, memory size, compute resources, volume mounts, and more. These two components live in the code ensuring reproduceable deployments.

Line 29 in file pipelines/inference/model_server.py on branch deployed_inference shows an example of a Modal image which states all required dependencies for the container.

Line 227 shows an example of a Modal function decorator in file modal/training.py on branch modal_integration_training.

2.4 Continuous Deployment using Github Actions

Given that we have an application service that should ideally always be available and functional to customers, we should (1) automate deployments directly after new (verified) code updates, and (2) create a separate testing service that mirrors prod behavior. We used Github Actions to deploy the Modal inference service when triggered by a push to the dev branch. Although not yet implemented fully, the dev or prod environment would also change the S3 bucket source and potentially model version to be a smaller, lighter version for testing.

This is shown in Figure 2 with the Github Actions (CD) tile looking to mediate the resources selected by the deployments. The matching code can be found on the .github/workflows/deploy.yml file in the main branch.

3 Disaster Recovery

https://drive.google.com/file/d/1P1gP6SYnOhbk7A1sH4mMIcQTLVwAKZ6z/view?usp=sharing